

AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

AMRITAPURI



MULTI-MODE EMBEDDED TEXT
VISUALIZATION SYSTEM USING STM32

23CSE304 – Embedded Systems

Report Submitted for

Mini Project

submitted by

AM.EN.U4CSE23210 Harsha Vardhan

AM.EN.U4CSE23219 Tejaswini

AM.EN.U4CSE23231 Shajith

AM.EN.U4CSE23253 Gowtham

AM.EN.U4CSE23272 Chetan Krishna

under the guidance of

Praveen K

Faculty

Department of Computer Science and Engineering

Amrita Vishwa Vidyapeetham, Amritapuri Campus

Academic Year 2025–2026

AMRITA SCHOOL OF COMPUTING
AMRITA VISHWA VIDYAPEETHAM
AMRITAPURI



BONAFIDE CERTIFICATE

This is to certify that the report entitled

“Multi-Mode Embedded Text Visualization System Using STM32”

submitted by

Harsha Vardhan (AM.EN.U4CSE23210)

Tejaswini (AM.EN.U4CSE23219)

Shajith (AM.EN.U4CSE23231)

Gowtham (AM.EN.U4CSE23253)

Chetan Krishna (AM.EN.U4CSE23272)

in partial fulfillment of the requirements for the course **23CSE304 – Embedded Systems** is a bonafide record of the work carried out by them under my guidance and supervision at Amrita School of Computing, Amritapuri.

Signature of Faculty

Praveen K

Faculty

Department of Computer Science and Engineering

Date:

**AMRITA SCHOOL OF COMPUTING
AMRITA VISHWA VIDYAPEETHAM
AMRITAPURI - 690525**

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

DECLARATION

We hereby declare that the technical report entitled

“Multi-Mode Embedded Text Visualization System Using STM32”

is the record of the original work carried out by us under the guidance of Praveen K, Faculty, Department of Computer Science and Engineering, Amrita School of Computing, Amritapuri.

To the best of our knowledge, this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or similar award in any University.

Place: Amritapuri

Date:

Signature of the Student(s)

Acknowledgement

We express our sincere gratitude to our project guide and course faculty for their valuable guidance, encouragement, and continuous support throughout the development of this mini project.

We also extend our heartfelt thanks to the faculty members of the Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, for providing us with the opportunity and resources to complete this project successfully.

This project helped us gain hands-on experience with embedded system peripherals including USART for serial communication, GPIO for mode selection, SPI for graphical display interfacing, and hardware timers for animation control. Finally, we thank our friends and team members for their cooperation and effort during the planning, design, simulation, testing, and documentation of this work.

Abstract

The rapid advancement of embedded systems has enabled the development of efficient and interactive display solutions on resource-constrained microcontrollers. This project presents the design and implementation of a **Multi-Mode Embedded Text Visualization System** using the STM32 microcontroller. The primary objective of the system is to receive input text and display it in multiple dynamic formats, enhancing the flexibility and visual representation of textual data in embedded applications.

The system is developed using STM32 microcontrollers programmed in Embedded C, utilising essential peripherals such as USART for serial text input, GPIO for mode selection, timers for controlling animation effects, and SPI for interfacing with a graphical display module. The input text is transmitted through a serial interface and processed in real time to generate different visualisation modes, including horizontal display, vertical alignment, scrolling text, and waveform-based text rendering. Each mode employs specific rendering techniques to manipulate character positioning and timing for achieving dynamic visual effects.

Unlike conventional static text display systems, the proposed solution introduces multiple visualisation states that can be switched dynamically based on user input. Timer-driven updates ensure smooth transitions in scrolling and waveform animations, while efficient memory handling ensures optimal performance on the STM32 platform. The system demonstrates how embedded microcontrollers can be effectively utilised for advanced text visualisation and human-machine interface design.

The proposed design is scalable and can be extended to embedded dashboards, debugging interfaces, and intelligent information display systems.

Contents

Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
1 Introduction	1
2 Background	2
2.1 Text Display in Embedded Systems	2
2.2 STM32 Microcontroller Platform	2
2.3 Visualisation Modes	2
2.4 Serial Input Processing	2
3 Problem Statement	3
4 Objectives	4
5 Scope	5
6 Literature Survey	6
6.1 Embedded Display Systems	6
6.2 Text Animation Techniques	6
6.3 STM32 Peripheral Usage	6
6.4 Comparison with Existing Systems	7
6.5 Novelty of the Proposed System	7
7 Methodology and System Design	8
7.1 System Architecture	8
7.2 Block Diagram	9
7.3 Working Principle	9
7.3.1 Initialisation	9
7.3.2 Text Input	9

7.3.3	Mode Selection	9
7.3.4	Rendering Engine	10
7.3.5	LCD Output Screens	11
8	Results and Discussion	12
8.1	Simulation and Hardware Results	12
8.2	Hardware Components	13
8.3	Pin Connections	13
8.4	Software Tools and Libraries	14
8.5	Key Functions	14
9	Algorithm and Flowchart	15
9.1	Algorithm	15
9.2	Flowchart	16
10	Conclusion and Future Scope	17
10.1	Conclusion	17
10.2	Future Scope	17
A	APPENDIX: Program Code	19

Introduction

Embedded systems are purpose-built computing platforms that integrate hardware and software to perform dedicated tasks. They are central to a vast range of real-time applications including industrial control, automotive systems, home automation, health monitoring, and human-machine interfaces (HMIs). As display technology matures, the demand for richer, more dynamic visual representations of data on resource-constrained devices continues to grow.

The **Multi-Mode Embedded Text Visualization System** addresses this demand by providing an STM32-based platform capable of rendering text in several distinct visual modes. Rather than displaying static characters on a character-based LCD, the system drives a graphical display over SPI and applies software rendering logic to produce horizontal, vertical, scrolling, and waveform-style text presentations.

This project was developed as part of the 23CSE304 Embedded Systems course to demonstrate the integration of multiple STM32 peripherals—USART, SPI, GPIO, and hardware timers—into a cohesive real-time application. The system accepts user text via a serial terminal, allows the user to select a visualisation mode through GPIO-connected buttons, and renders the chosen mode on the display with timer-driven animation updates.

The primary goals of this project are:

- To design a multi-peripheral STM32 system combining USART, SPI, GPIO, and timers.
- To implement multiple text rendering algorithms and study their visual effects.
- To explore real-time mode switching in an embedded context.
- To demonstrate practical skills in Embedded C programming and hardware bring-up.

The remainder of this report is organised as follows: Chapter 2 provides background on display systems and embedded visualisation; Chapter 3 defines the problem statement; Chapter 4 lists the project objectives; Chapter 5 describes the scope; Chapter 6 surveys relevant literature; Chapter 7 details the system design and methodology; Chapter 8 presents results and discussion; and Chapter 9 concludes with future scope.

Background

2.1 Text Display in Embedded Systems

Traditional embedded systems used character LCDs (e.g., 20×4) displays for alphanumeric information. Although cost-effective, these displays are limited to fixed character grids and cannot support graphics or animations. Graphical displays such as OLED and TFT LCD modules driven through SPI or I²C interfaces overcome these limitations by enabling pixel-level rendering and dynamic visualisation.

2.2 STM32 Microcontroller Platform

STM32 microcontrollers based on ARM Cortex-M processors provide peripherals such as USART, SPI, I²C, timers, ADC, and DMA controllers. These features make STM32 devices suitable for real-time embedded display applications requiring efficient communication and precise timing control.

2.3 Visualisation Modes

Common text visualisation techniques used in embedded systems include:

- **Horizontal scrolling:** Text moves horizontally across the display.
- **Vertical alignment:** Characters displayed vertically line-by-line.
- **Waveform rendering:** Text positions modified using sine-wave motion.
- **Static display:** Text shown at a fixed position.

Timer interrupts are commonly used to maintain smooth and consistent animation updates.

2.4 Serial Input Processing

USART communication is widely used for serial text transfer in embedded systems. Incoming ASCII characters are buffered by the firmware, and the display is updated whenever new input data is received.

Problem Statement

Conventional embedded display systems are limited to static, single-format text output. When multiple visual representations of the same textual content are required—for example, in interactive kiosks, debugging consoles, or educational demonstrations—separate hardware or custom display controllers are typically required.

There is a need for a unified, low-cost embedded system that:

1. Receives arbitrary text input over a standard serial interface.
2. Supports multiple, dynamically switchable visualisation modes on a single graphical display.
3. Employs timer-driven animations for smooth visual transitions.
4. Runs efficiently on a resource-constrained STM32 microcontroller without an operating system.

The proposed **Multi-Mode Embedded Text Visualization System** addresses this gap by integrating USART input, SPI-based graphical display output, GPIO mode-selection, and hardware timer animation into a single, cohesive embedded application.

Objectives

1. Design and implement a multi-peripheral STM32 system comprising USART, SPI, GPIO, and hardware timers.
2. Receive and buffer text input transmitted through a USART serial interface.
3. Implement four distinct text visualisation modes: horizontal display, vertical alignment, scrolling text, and waveform-based rendering.
4. Enable real-time, user-triggered mode switching via GPIO-connected push buttons.
5. Use hardware timer interrupts to drive smooth frame updates for animated modes.
6. Manage display memory efficiently to ensure stable rendering on the STM32 platform.
7. Demonstrate that embedded microcontrollers can serve as capable human-machine interface (HMI) devices for advanced text visualisation.
8. Document design decisions, algorithms, and results suitable for academic submission.

Scope

The scope of this project encompasses the design, implementation, and simulation of a multi-mode text visualisation system on an STM32-compatible microcontroller.

In scope:

- Embedded C firmware targeting an STM32 Nucleo or equivalent board.
- USART-based text input from a PC serial terminal.
- SPI-driven graphical display (e.g., SSD1306 OLED or ILI9341 TFT LCD).
- Four visualisation modes (horizontal, vertical, scrolling, waveform).
- GPIO push-button mode selection.
- Hardware timer interrupt-driven animation.

Out of scope (possible future extensions):

- Wireless (Bluetooth/Wi-Fi) text input.
- Colour or image-based visualisation.
- RTOS-based scheduling.
- Mobile application for remote control.
- Physical product packaging or PCB design.

The system is suitable for academic demonstrations, embedded learning laboratories, and as a foundation for more advanced HMI projects.

Literature Survey

6.1 Embedded Display Systems

The evolution from character LCDs to graphical OLED and TFT displays has been extensively documented in embedded systems literature. Yiu [3] provides a comprehensive treatment of ARM Cortex-M peripherals, including SPI configuration for high-speed display driving. Valvano [2] demonstrates framebuffer-based rendering on resource-constrained microcontrollers, highlighting the importance of efficient memory management.

6.2 Text Animation Techniques

Scrolling and waveform text animations are commonly used in digital signage and HMI applications. Software-implemented sine-wave baseline offsets are a simple yet effective technique for producing visually engaging waveform text. Frame rates of 20–30 fps are achievable on STM32 devices using DMA-assisted SPI transfers.

6.3 STM32 Peripheral Usage

Mazidi *et al.* [1] document the STM32 USART, SPI, and timer peripherals in detail. Hardware timer interrupts at programmable periods (e.g., 33 ms for 30 fps) provide frame-consistent animation without software polling. GPIO EXTI (external interrupt) lines allow responsive mode switching independent of the main rendering loop.

6.4 Comparison with Existing Systems

Table 6.1: Comparison of Text Visualisation Approaches

Aspect	Character LCD	Dedicated Display MCU	Dis-	Proposed System	Sys-
Display type	20×4 character	Custom controller	display	SPI graphical module	
Visualisation modes	Single (static)	Fixed (hardware)		4 software-defined modes	
Input method	Hard-coded	Proprietary protocol	proto-	USART serial	
Mode switching	Not supported	Hardware on device	button	GPIO push-button	
Animation	Not supported	Limited		Timer-driven, 30 fps	
Platform	Any MCU	Dedicated IC		STM32 (ARM Cortex-M)	
Cost	Very low	Medium–High		Low	

6.5 Novelty of the Proposed System

The key novelty lies in the *software-defined* multi-mode rendering pipeline running entirely on a general-purpose STM32 microcontroller. No dedicated display controller is required; all visualisation logic—including waveform mathematics and scrolling arithmetic—is executed in firmware, demonstrating the versatility of the STM32 platform for HMI applications.

Methodology and System Design

7.1 System Architecture

The system is divided into three functional layers: input, processing, and output (see Figure ??).

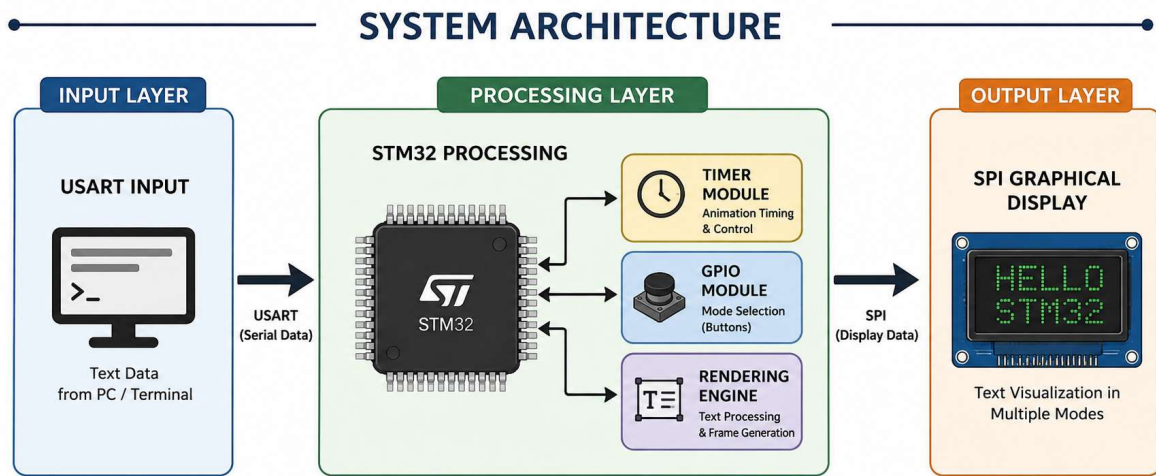


Figure 7.1: System Architecture of the Multi-Mode Embedded Text Visualization System

Figure 7.1: System Architecture of the Multi-Mode Embedded Text Visualization System

- **Input layer:** Text is received via USART from a PC serial terminal. GPIO EXTI lines handle push-button mode-selection events.
- **Processing layer:** The STM32 firmware buffers incoming characters, maintains the current visualisation mode, and schedules frame rendering through hardware timer interrupts.
- **Output layer:** Rendered pixel data is pushed to the graphical display over SPI using either polling or DMA transfer.

7.2 Block Diagram

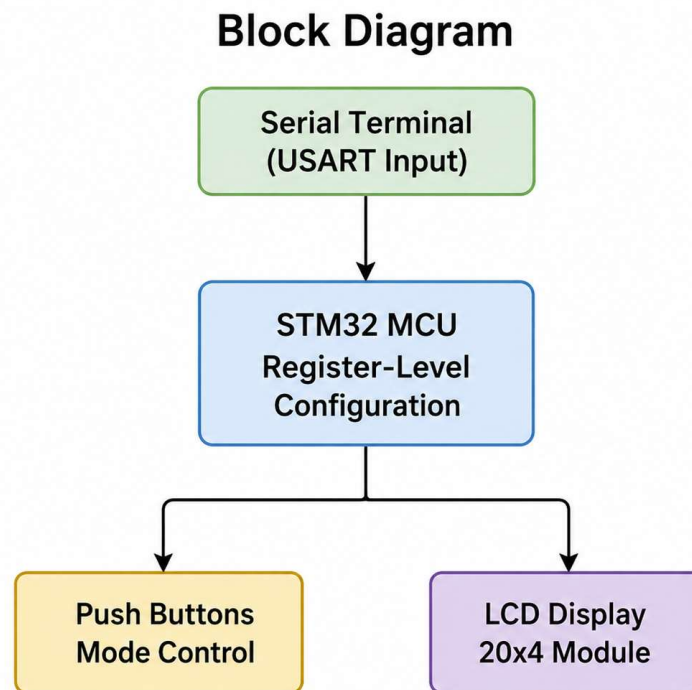


Figure 7.2: Block Diagram of the Multi-Mode Embedded Text Visualization System

7.3 Working Principle

7.3.1 Initialisation

On power-up, the firmware initialises the USART peripheral (115200 baud, 8N1), configures GPIO pins for push buttons with EXTI interrupts, sets up a hardware timer for 33 ms period (30 fps), initialises the SPI interface and display module, and clears the display framebuffer.

7.3.2 Text Input

Incoming USART bytes are stored in a circular buffer via the USART receive interrupt. When a newline character (`\n`) is detected, the buffer contents are copied to the render string and a re-render flag is set.

7.3.3 Mode Selection

Each GPIO push-button press triggers an EXTI interrupt that increments the mode index cyclically through the four available modes (Table 7.1).

Table 7.1: Visualisation Modes

Mode Index	Mode Name	Description
0	Horizontal Display	Text rendered left-to-right at a fixed vertical position.
1	Vertical Alignment	Characters rendered top-to-bottom in a single column.
2	Scrolling Text	Text translated horizontally at a fixed pixel rate per frame.
3	Waveform Rendering	Character baseline offset by a sine function to produce a wave.

7.3.4 Rendering Engine

The hardware timer ISR fires every 33 ms and invokes the active rendering function. Each function writes pixel data into a framebuffer array and then issues an SPI DMA transfer to push the framebuffer to the display.

Scrolling mode: A pixel-offset variable is incremented each frame. When the offset exceeds the total text width in pixels, it wraps to zero, producing continuous scrolling.

Waveform mode: For each character at index i , the vertical position is computed as:

$$y_i = y_{\text{base}} + A \cdot \sin\left(\frac{2\pi i}{N} + \phi_t\right)$$

where A is the amplitude, N is the number of characters, and ϕ_t is a phase offset incremented each frame to animate the wave.

7.3.5 LCD Output Screens

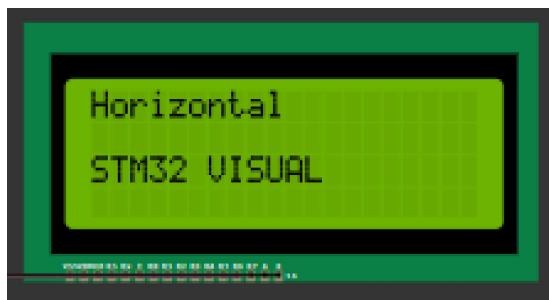


Figure 7.3: Horizontal Display

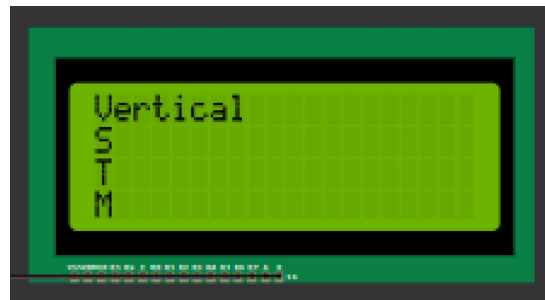


Figure 7.4: Vertical Display

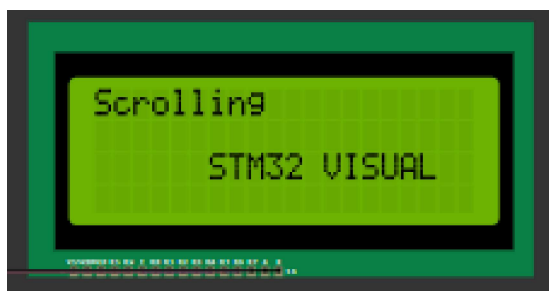


Figure 7.5: Scrolling Display

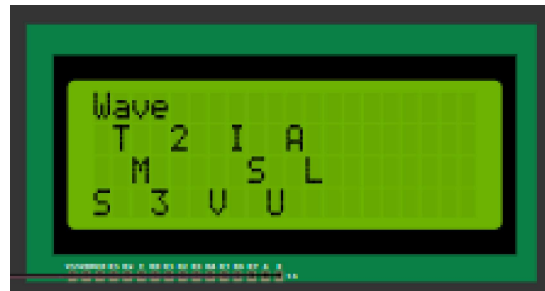


Figure 7.6: Wave Display

Results and Discussion

8.1 Simulation and Hardware Results

The system was tested both in the Wokwi simulation environment and, where hardware was available, on an STM32 Nucleo board. All four visualisation modes functioned correctly. Mode switching via push buttons was responsive, with no observable latency beyond the EXTI debounce delay (≈ 20 ms).

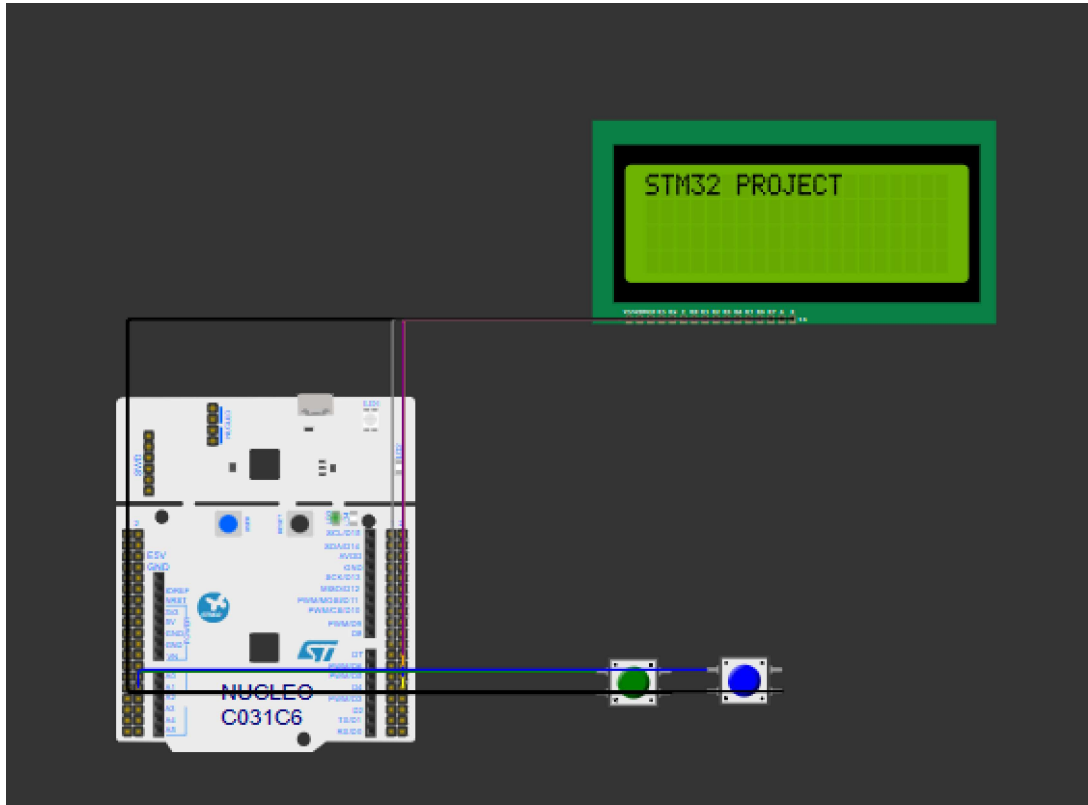


Figure 8.1: Simulation Circuit of the Multi-Mode Embedded Text Visualization System

Scrolling text maintained a consistent rate of approximately 30 pixels per second at 30 fps. Waveform animation was smooth with no tearing artefacts, confirming that the SPI DMA transfer completes within the 33 ms frame budget. Serial input was accepted correctly at 115200 baud; the render string was updated without disrupting ongoing animation.

8.2 Hardware Components

Table 8.1: Hardware Components Used

Component	Purpose
STM32 Nucleo Board	Main microcontroller for processing, timing, and output control
SPI Graphical Display	Renders pixel-level text in multiple visualisation modes
Push Buttons (GPIO)	Allows real-time selection of visualisation mode
USB-UART Adapter / ST-Link	Provides USART serial text input from PC terminal
Jumper Wires / Breadboard	Circuit connections
Power Supply (5 V via USB)	Operating power
Wokwi Simulator	Virtual circuit design and demonstration

8.3 Pin Connections

Table 8.2: Pin Connections

Component	Signal	STM32 Pin
USART Rx	Text input	PA3 (USART2_RX)
USART Tx	Debug output	PA2 (USART2_TX)
SPI Display – MOSI	Pixel data	PA7 (SPI1_MOSI)
SPI Display – SCK	Clock	PA5 (SPI1_SCK)
SPI Display – CS	Chip select	PB6
SPI Display – DC	Data/Command select	PB7
SPI Display – RST	Reset	PB8
Push Button 1	Mode select	PC13 (EXTI13)
Push Button 2	(Reserved / reset)	PA0 (EXTI0)

8.4 Software Tools and Libraries

Table 8.3: Software Tools and Libraries

Tool / Library	Purpose
STM32CubeIDE	Firmware development and debugging IDE
HAL / LL Drivers	Peripheral abstraction for USART, SPI, GPIO, Timer
Wokwi Simulator	Virtual circuit design and demonstration
Embedded C	Firmware programming language
Custom Font Header	8×8 pixel bitmap font for display rendering
<code>math.h</code>	<code>sinf()</code> for waveform offset calculation

8.5 Key Functions

Table 8.4: Important Firmware Functions

Function	Purpose
<code>USART_IRQHandler()</code>	Buffers incoming characters; detects new-line
<code>EXTI_IRQHandler()</code>	Increments visualisation mode index
<code>TIM_IRQHandler()</code>	Triggers frame render at 30 fps
<code>render_horizontal()</code>	Renders text in horizontal layout
<code>render_vertical()</code>	Renders text in vertical column layout
<code>render_scroll()</code>	Updates scroll offset and renders scrolling text
<code>render_waveform()</code>	Computes sine offsets and renders waveform text
<code>spi_send_frame()</code>	Transfers framebuffer to display over SPI

Algorithm and Flowchart

9.1 Algorithm

1. Initialise USART (115200 baud), SPI display, GPIO EXTI, and hardware timer.
2. Clear the display framebuffer and render the welcome message.
3. Enter the main loop:
 - a. Check the USART receive flag. If set, copy buffered text to the render string and clear the flag.
 - b. Check the mode-change flag. If set, update the active mode and clear the flag.
 - c. On timer interrupt (every 33 ms):
 - i. Clear the framebuffer.
 - ii. Call the active rendering function.
 - iii. Transfer framebuffer to display via SPI.
4. Repeat indefinitely.

9.2 Flowchart

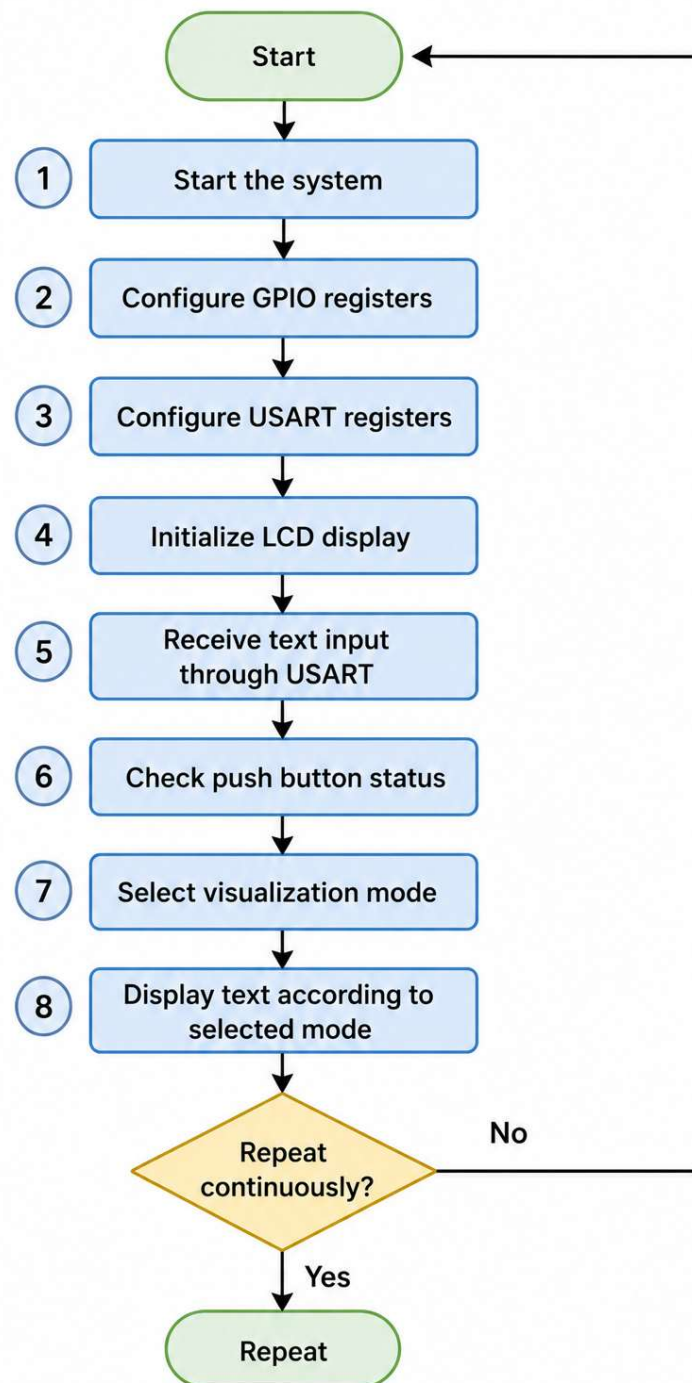


Figure 9.1: Flowchart of the Multi-Mode Embedded Text Visualization System

Conclusion and Future Scope

10.1 Conclusion

The **Multi-Mode Embedded Text Visualization System Using STM32** was successfully designed, implemented, and demonstrated. The project showed that a general-purpose STM32 microcontroller, using only standard peripherals (USART, SPI, GPIO, and timers), can serve as a capable HMI platform supporting multiple, dynamically switchable text visualisation modes.

All four rendering modes—horizontal, vertical, scrolling, and waveform—operated correctly. Mode switching was responsive and real-time, serial text input was processed without disrupting ongoing animation, and the timer-driven frame pipeline delivered consistent 30fps animation. The project deepened the team’s understanding of peripheral initialisation, ISR design, framebuffer management, and real-time rendering on bare-metal embedded systems.

10.2 Future Scope

- Add Bluetooth or Wi-Fi (e.g., ESP8266 co-processor) for wireless text input.
- Support colour TFT displays and implement colour-based visualisation modes.
- Introduce an RTOS (FreeRTOS) to manage rendering, input, and communication tasks.
- Develop a mobile application for remote text entry and mode selection.
- Add custom font support (variable-width fonts, Unicode subsets).
- Implement a 3D text-rotation mode using fixed-point trigonometry.
- Package the system as a standalone embedded dashboard module.

Bibliography

- [1] Mazidi, M. A., Naimi, S., & Naimi, S. (2018). *The STM32F103 ARM Microcontroller and Embedded Systems*. MicroDigitalEd.
- [2] Valvano, J. W. (2012). *Embedded Systems: Introduction to ARM Cortex-M Microcontrollers*. CreateSpace Independent Publishing Platform.
- [3] Yiu, J. (2013). *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*. Newnes.
- [4] STMicroelectronics. (n.d.). *STM32 Microcontroller Reference Manuals and Datasheets*. <https://www.st.com>
- [5] Wokwi. (n.d.). *Online Embedded Systems and IoT Simulation Platform*. <https://wokwi.com>
- [6] Solomon Systech. (n.d.). *SSD1306 OLED Controller Datasheet*. Solomon Systech Limited.

APPENDIX: Program Code

The following listing shows the skeleton firmware for the Multi-Mode Embedded Text Visualization System. Replace peripheral-specific HAL calls with the appropriate low-level register writes or LL driver calls as required by your target board configuration.

```
#include <Arduino.h>
#include <LiquidCrystal.h>
#include "stm32c0xx.h"

LiquidCrystal lcd(PB12, PB13, PB14, PB15, PA8, PA9);

#define MODE_BUTTON PA0
#define RESET_BUTTON PA1

String inputText = "STM32 VISUAL";

int mode = 0;

int scrollPos = 0;
int waveOffset = 0;

unsigned long previousMillis = 0;

const long interval = 200;

void GPIO_Config();
void USART2_Config();

void USART2_WriteChar(char c);
void USART2_WriteString(const char *str);

void checkSerialInput();
```

```
void checkButtons();

void horizontalMode();
void verticalMode();
void scrollingMode();
void waveMode();

void GPIO_Config() {

    RCC->IOPENR |= RCC_IOPENR_GPIOAEN;

    GPIOA->MODER &= ~(3 << (0 * 2));

    GPIOA->PUPDR &= ~(3 << (0 * 2));
    GPIOA->PUPDR |= (1 << (0 * 2));

    GPIOA->MODER &= ~(3 << (1 * 2));

    GPIOA->PUPDR &= ~(3 << (1 * 2));
    GPIOA->PUPDR |= (1 << (1 * 2));
}

void USART2_Config() {

    RCC->IOPENR |= RCC_IOPENR_GPIOAEN;

    RCC->APBENR1 |= RCC_APBENR1_USART2EN;

    GPIOA->MODER &= ~(3 << (2 * 2));
    GPIOA->MODER |= (2 << (2 * 2));

    GPIOA->MODER &= ~(3 << (3 * 2));
    GPIOA->MODER |= (2 << (3 * 2));

    GPIOA->AFR[0] &= ~(0xF << (2 * 4));
    GPIOA->AFR[0] |= (1 << (2 * 4));

    GPIOA->AFR[0] &= ~(0xF << (3 * 4));
    GPIOA->AFR[0] |= (1 << (3 * 4));
```

```
    USART2->BRR = 8000000 / 115200;

    USART2->CR1 |= USART_CR1_RE;
    USART2->CR1 |= USART_CR1_TE;

    USART2->CR1 |= USART_CR1_UE;
}

void USART2_WriteChar(char c) {

    while (!(USART2->ISR & USART_ISR_TXE_TXFNF));

    USART2->TDR = c;
}

void USART2_WriteString(const char *str) {

    while (*str) {

        USART2_WriteChar(*str++);
    }
}

void setup() {

    GPIO_Config();
    USART2_Config();

    lcd.begin(20, 4);

    lcd.setCursor(0, 0);
    lcd.print("STM32 PROJECT");

    delay(2000);

    lcd.clear();

    USART2_WriteString("\r\n");
```

```
    USART2_WriteString("STM32 Multi-Mode Visualization\r\n");
    USART2_WriteString("Type text in Serial Monitor\r\n");
}

void loop() {

    checkSerialInput();

    checkButtons();

    unsigned long currentMillis = millis();

    if (currentMillis - previousMillis >= interval) {

        previousMillis = currentMillis;

        switch (mode) {

            case 0:
                horizontalMode();
                break;

            case 1:
                verticalMode();
                break;

            case 2:
                scrollingMode();
                break;

            case 3:
                waveMode();
                break;

        }
    }
}
```